

AttenPIM: Accelerating LLM Attention with Dual-mode GEMV in Processing-in-Memory

Liyan Chen[†], Dongxu Lyu[†], Zhenyu Li, Jianfei Jiang, Qin Wang, Zhigang Mao, Naifeng Jing^{*}

Department of Micro/Nano Electronics, Shanghai Jiao Tong University, Shanghai, China

{liyan.chen, sjtj}@sjtu.edu.cn

Abstract—Large Language Models (LLMs) have demonstrated unprecedented generative performance across a wide range of applications. While recent heterogeneous architectures attempt to address the memory-bound bottleneck from attention computations by processing-in-memory (PIM) offloading, they overlook two critical characteristics of attention GEMVs that distinguish them from traditional PIM scenarios: (1) dynamic matrix dimensions that scale with token length, and (2) distinct GEMV patterns between score computation ($Q \times K_t$) and context computation ($S \times V$). Existing PIM designs, employing either uniform or transposed computing modes, suffer from inefficiencies in newly generated element preparation or distinct GEMV execution. To address these limitations, we propose AttenPIM, a software-hardware co-design for efficient PIM-based attention acceleration. For bank-level execution, we propose dual-mode computing modes tailored for score and context computations with PIM-oriented data layouts and execution flows for KV storage, supported by a low-cost configurable per-bank PIM unit (PU). For system-level execution, we leverage token-level and head-level concurrency to ensure workload balance and maximize bank PU parallelism. Furthermore, dynamic allocation and kernel fusion methods are proposed to further minimize memory overhead. Experimental results demonstrate that AttenPIM achieves $1.13 \times$ – $5.26 \times$ speedup and reduces energy consumption by 17%–49% compared to two state-of-the-art PIM baselines.

I. INTRODUCTION

Large Language Models (LLMs) have been widely deployed across diverse applications [1], such as language understanding, content generation, and decision support. LLM inference demands immense amounts of compute and memory resources. While machine learning accelerators (e.g., GPU and TPU, referred to as xPU) demonstrate high performance on compute-bound layers, they struggle with memory-bound layers due to limited memory bandwidth and resulting underutilization [2]–[4]. For instance, with a batch size of 16 and a token length of $2K$, the memory-bound multi-head attention (hereafter attention) layers of GPT-3 13B consume more than 50% of the total time on A100 GPUs [2]. This proportion increases dramatically with larger batch sizes or longer sequence lengths, severely limiting the inference performance [4].

The attention layer comprises three operations: score, softmax, and context, with score and context computations being the most time-consuming matrix-vector multiplication (GEMV) computations. Recent studies [5]–[8] offload attention operations to processing-in-memory (PIM) designs, which

have demonstrated significant performance improvements in GEMV computation [9], [10]. By integrating PIM units (PUs) within memory devices, PIM can achieve higher aggregated bandwidth and computational parallelism [11]–[14].

However, attention GEMVs in LLMs exhibit distinct characteristics that differentiate them from conventional PIM applications with static weight data and uniform execution flow [9], [10]. First, the dimensions of attention GEMV matrices dynamically expand with increasing token length, requiring additional matrix data preparation for each new token. Second, attention score and context computations, with transposed matrix dimensions, perform multiply-accumulate (MAC) operations along different dimensions during execution.

Existing PIM works fail to consider these characteristics with the PIM computing paradigm simultaneously, thus leading to performance degradation. NeuPIMs [6], built based on Newton [9], partitions both matrices column-wise while broadcasting vector data for parallel execution. However, the newly generated matrix elements cannot fully utilize DRAM burst transfer (16 bits out of 256/512 bits), incurring $50 \times$ preparation time with masked accesses [7]. While AttAcc [5] proposes a multi-level partition method for two matrices in a transposed manner, it requires PUs at multiple memory levels for partial result accumulation. Additionally, the inter-PU communication would congest memory buses (particularly TSVs), leading to 15%–58% performance degradation.

We identify that it is crucial to simultaneously address both attention GEMV characteristics and PIM computing paradigm design. To achieve this goal, we propose AttenPIM, a software-hardware co-design for efficient PIM-based attention acceleration. Our contributions are as follows:

- For bank-level execution, we propose dual computing modes for score and context computations with PIM-oriented data layouts and execution flow, supported by a low-cost configurable per-bank PIM unit (PU).
- For system-level execution, we leverage token-level and head-level concurrency to ensure workload balance and maximize bank PU parallelism. Furthermore, we introduce dynamic allocation and kernel fusion methods to reduce memory space waste and memory accesses.
- We implement our design on TSMC 28nm technology and extensively evaluate it on real-world LLM datasets. AttenPIM achieves a $1.13 \times$ – $5.26 \times$ speedup and reduces energy consumption by 17%–49% compared to two state-of-the-art PIM baselines.

This work was supported by National Natural Science Foundation of China (Grant U24B20165 and No.92264108).

[†] Co-first Author. ^{*} Corresponding Author.

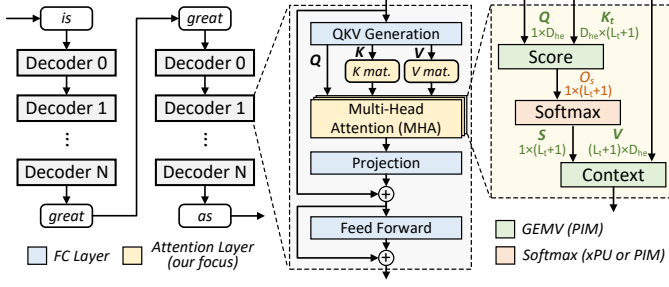


Fig. 1. Model architecture and inference (Gen stages) in LLMs.

II. BACKGROUND AND MOTIVATION

A. LLM Model

LLM models are built from the Transformer decoder [15], as illustrated in Fig. 1. Each decoder consists of fully-connected layers (QKV Generation, FFN), attention layers (MHA). The inference process consists of an initial Sum stage followed by recurring Gen stages. The Sum stage computes the initial KV cache (matrices) from the input prompt and generates the first token while each Gen stage updates KV matrices by concatenating and generates a new token auto-regressively.

The MHA layer consists of multiple independent heads, each performing two types of GEMVs: score computation ($Q \times K_t$) and context computation ($S \times V$), and a softmax operation. Given head embedding dimension D_{he} and current sequence length L_t , the score computation involves a $1 \times D_{he}$ vector Q and a $D_{he} \times (L_t + 1)$ matrix K_t , while the context computation involves a $1 \times (L_t + 1)$ vector S and a $(L_t + 1) \times D_{he}$ matrix V . The MHA layer with low compute-memory ratio is underutilized in xPU. For instance, with GPT-3 13B under a batch size of 16 and token length of $2K$, it can consume more than 50% of the total time on A100 GPUs [4].

B. HBM-based PIM for Attention

An HBM device [16] integrates multiple DRAM dies with a buffer die through TSVs, as shown in Fig. 2(a). Each DRAM die comprises independent (pseudo) channels, with each channel subdivided into multiple bank groups (BGs) consisting of four banks. HBM-based PIM, which incorporates PUs at various memory hierarchies (die, BG, bank), offers promising solutions for GEMV through high aggregated bandwidth.

Recent studies have explored offloading attention GEMVs to HBM-based PIM for acceleration [5], [6]. NeuPIMs [6] and works [7], [8], depicted in Fig. 2(b), partitions both matrices K_t and V column-wise. They leverage a global shared buffer to broadcast vector data while confining GEMV execution within bank PUs. AttAcc [5], as shown in Fig. 2(c), partitions matrix K_t token-wise across BGs, with each matrix tile further subdivided embedding-wise within each BG, and leverages a transposed layout for matrix V . Partial results of $Q \times K_t$ in bank PUs are accumulated in BG PUs, while partial results of $S \times V$ are accumulated in die PU.

Considering that softmax operation execution time is typically masked through head-level pipelining [5], [6], this paper primarily concentrates on attention GEMVs. Notably,

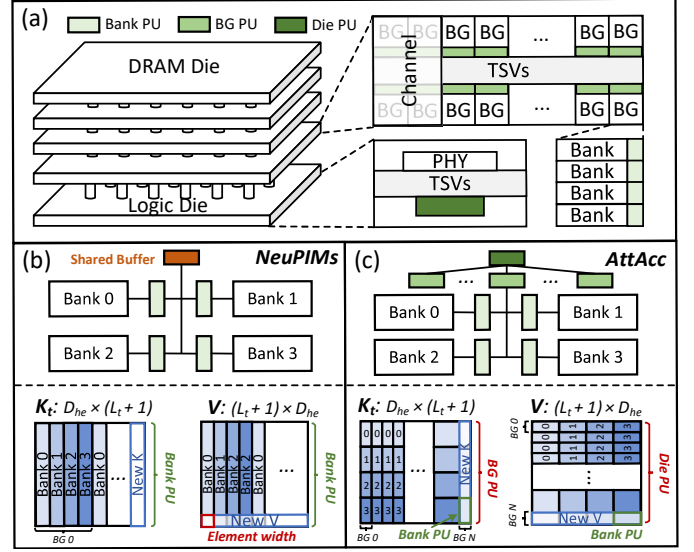


Fig. 2. (a) HBM-PIM. (b) NeuPIMs architecture. (c) AttAcc architecture.

AttenPIM design can also extend to PIM designs based on other memory devices, such as GDDR [10], DIMM [17].

C. Motivation

Attention GEMVs in LLMs exhibit unique characteristics that distinguish them from conventional PIM applications [9], [10]. First, attention matrices expand with token length, introducing an additional matrix preparation phase. Second, attention GEMVs encompass score and context computations with transposed matrix layouts, which perform MAC operations along different dimensions during execution. The efficiency of both phases is fundamentally determined by matrix data layout. While existing PIM designs adopt either uniform or transposed partitioning strategies to distribute matrices K and V , these one-size-fits-all approaches demonstrate inefficiencies in either the preparation or computation phase.

NeuPIMs distributes newly generated V head elements across different banks, where the element width (e.g., 16 bits) underutilizes burst transfer capabilities (e.g., 256/512 bits). The masked accesses for V preparation consume about $50\times$ more time compared with preparation for matrix K_t , accounting for 12%-77% of total execution time across various token lengths. AttAcc introduces contention on shared memory buses between PUs. Specifically, TSV bandwidth emerges as a bottleneck for context computation ($S \times V$), resulting in 15%-58% performance degradation. Additionally, the requirement for diverse vector data across bank PUs necessitates distributed input buffers in each bank PU. These performance penalties accumulate during token generation, substantially impacting end-to-end inference performance.

Motivated by these observations, we identify that there is an urgent need for specialized data layouts and compatible execution flows tailored separately for PIM-based attention score computation ($Q \times K_t$) and context computation ($S \times V$), which can maintain burst transfer efficiency and eliminate inter-PU communication overhead.

III. ATTENPIM ARCHITECTURE

AttenPIM implements a multi-level acceleration strategy for LLM attention, spanning from bank-level optimizations to system-level enhancements. At the bank level, we introduce dual computing modes for score and context computations with PIM-oriented data layouts and execution flows. We confine GEMV execution within bank PUs while leveraging broadcast mechanisms to eliminate distributed buffers. At the system level, we exploit token-level parallelism and head-level parallelism to ensure workload balance, complemented by kernel fusion and dynamic allocation methods to minimize memory accesses and memory waste. Given the independent nature of attention head computations, we use a single head as a representative example, unless stated otherwise.

A. Computing Mode: Score

Building upon the demonstrated efficiency of NeuPIMs in score computation ($Q \times K_t$) with bank PUs, we follow their proposed data layout and execution flow. We present a brief overview using a two-bank configuration as an illustrative example, while not claiming this as our innovation.

1) *Data Layout*: As illustrated in Fig. 3(a), when a new K_t column is generated, all head elements are mapped to a designated bank and stored continuously within a bank row. In this case, score computation ($Q \times K_t$) can be performed within bank PU with high DRAM operation efficiency during both the storage and execution stage.

2) *Execution Flow*: Inner-product is leveraged for score computation ($Q \times K_t$). As depicted in Fig. 3(c), head elements from two tokens are positioned in identical row and column locations across different banks to enable parallel read operations. In each phase with burst-width transfer, bank PUs retrieve head elements from DRAM arrays (①), while the shared buffer broadcasts corresponding vector data of Q to all bank PUs (②). These elements undergo multiplication and accumulation to generate a partial output element (③). The output elements from different phases are accumulated to generate the final result (④).

B. Computing Mode: Context

AttenPIM introduces a specialized data layout for V matrix storage and implements a compatible execution flow for context computation ($S \times V$).

1) *Data Layout*: We identify burst length as the minimum storage granularity that maintains DRAM operation efficiency, which we leverage for partitioning each newly generated vector V . Consider the burst transfer length of N_{bl} and model element precision of N_{pre} . As shown in Fig. 3(b), a token head is divided into N_{BB} burst blocks, where $N_{BB} = D_{he}/N_{bl}$. Each burst block consists of N_{bl}/N_{pre} elements. The mapping of these burst blocks follows two key principles: (1) burst blocks with identical indices across different tokens reside in the same bank (e.g., BB_1 of all tokens are stored continuously in bank 1), and (2) data alignment in identical rows and columns is maintained to enable parallel read operations between banks.

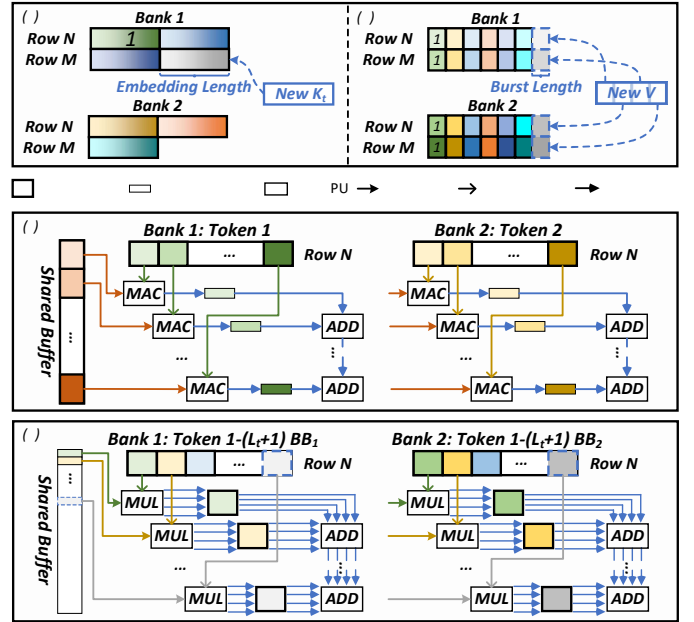


Fig. 3. (a) Data layout for K matrix. (b) Data layout for V matrix. (c) Execution flow for score. (d) Execution flow for context.

2) *Execution Flow*: The data layout ensures that elements requiring multiplication and addition operations are localized within the same bank. Based on it, we propose to perform context computation ($S \times V$) using an outer product approach. As depicted in Fig. 3(d), burst blocks BB_1 and BB_2 of N_t token are allocated to bank 1 and bank 2, respectively. During each phase, bank PUs read a burst block of a token from DRAM arrays (⑤), while the shared buffer only needs to broadcast a single element of vector S to bank PUs (⑥). This element is shared and multiplied with each head element of the burst block instead of MAC, generating N_{bl}/N_{pre} partial outputs (⑦). These partial results undergo element-wise accumulation across phases to generate the final results (⑧).

C. Hardware Support: Configurable PU

To efficiently support dual-mode GEMV operations, we propose a cost-effective configurable PIM unit at the bank level, complemented by a global shared buffer for data broadcasting. To match the burst transfer bandwidth, each PU incorporates N_{bl}/N_{pre} multipliers and adders, along with multiple buffers for pipelined execution.

1) *Inner product*: The inner product computation follows an adder tree architecture, as illustrated in Fig. 4(a). Using a configuration with four multipliers, four adders, and a result buffer as an example: during each phase, the multipliers receive and execute burst-length inputs of N_{bl}/N_{pre} elements from both the shared buffer and DRAM row buffer. The adder tree performs accumulation of outputs from multipliers and accumulation of partial outputs between phases. Consequently, $D_{he} \times N_{pre}/N_{bl}$ phases are needed to complete one head GEMV. The host can either read the result buffer for softmax operation (kernel fusion detailed in Section III-E2) or write it back to banks for subsequent computations.

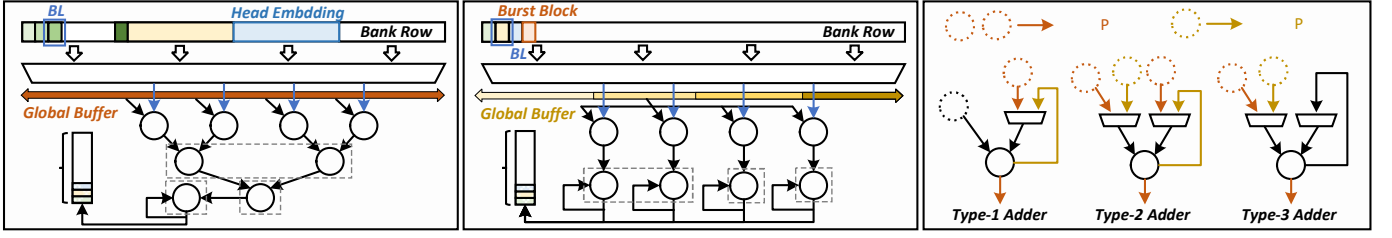


Fig. 4. (a) Inner product mode for score. (b) Outer product mode for context. (c) Three types of adders. Buffers for pipeline are not shown.

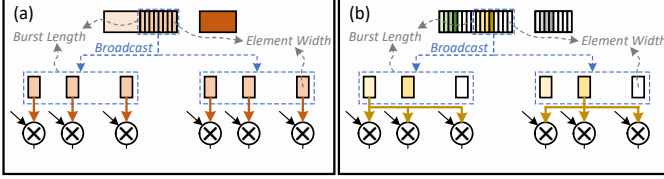


Fig. 5. (a) Broadcast in score execution. (b) Broadcast in context execution.

2) *Outer product*: The outer product implementation employs a self-accumulation approach, where outputs accumulate within individual multiplier-adder sets without inter-set data transfer. As shown in Fig. 4(b), in each phase, a single input element from the shared buffer is shared by all multipliers to be multiplied with data from DRAM row buffer. Each set's adder combines the new multiplier output with the previously accumulated sum. In this case, N_{bl}/D_{he} outputs are generated concurrently through self-accumulating. L_t+1 phases are needed to generate final results.

3) *Configurable Path*: The Bank PU implements three types of adders to support both inner and outer product operations, as depicted in Fig. 4(c). First, the type-1 adder combines input from a multiplier with either another multiplier's output (inner product) or its own previous output (outer product). Second, the type-2 adder processes inputs from two other adders (inner product) or combines a multiplier output with its own previous output (outer product). Third, the type-3 adder adds its previous output with another adder's output (inner product) or a multiplier's output (outer product). The mode configuration is performed only once for each head of the entire batch to amortize overhead.

D. Workload Balance

1) *Score $Q \times K_t$* : AttenPIM adopts the mapping strategy in NeuPIMs, assigning each head of K matrix to one channel and each token to one bank. In contrast, AttAcc maps a head across the entire HBM device and allocates each token to a BG for cross-level accumulation. Since the number of BGs in an HBM device significantly exceeds the number of banks in a channel, NeuPIMs and AttenPIM achieve better workload balance for requests with relatively short token lengths.

2) *Context $S \times V$* : The context execution of one head cannot exploit all-bank parallelism, thus leading to potential workload imbalance. Specifically, the number of burst block N_{BB} is typically smaller than the available channel banks N_{bank} (e.g., $N_{BB}=8$ in GPT-3 while $N_{bank}>16$ in a HBM device). To

address this limitation, we implement concurrent context computation for multiple heads instead of single-head execution across all banks.

For head selection, we prioritize different heads from a single request over heads from different requests for two key reasons: (1) Requests with varying token lengths require different numbers of phases for context completion, potentially causing new workload imbalance. (2) Data arrangement in banks may differ across requests, preventing effective parallel read operations. Conversely, heads within the same request maintain consistent phase length and data layout. Consequently, we perform context execution for N_{bank}/N_{BB} heads in a channel at the same time.

Unlike score computation ($Q \times K_t$) where all bank PUs share the same input vector Q , context matrices V of different heads multiply with different input vectors S , challenging the broadcast mechanism. However, in our computing mode for context, only one element is required in the execution phase of N_{BB} bank PUs (one head), allowing the remaining bits of the shared buffer to be leveraged for other heads. As illustrated in Fig. 5(a), for score computation, the elements broadcast by the shared buffer are leveraged in all banks, multiplied with corresponding matrix data. In contrast, as shown in Fig. 5(b), different elements from the shared buffer are leveraged in context computations of different heads. Consequently, the shared buses can provide vector S elements to N_{bl}/N_{pre} heads' context computations concurrently, with each head utilizing N_{BB} banks, thereby ensuring optimal all-bank parallelism and workload balance.

E. Memory Optimization

1) *Dynamic Allocation*: Recent research has introduced dynamic KV allocation strategies (e.g., allocating memory space for several tokens when needed) to decrease memory waste from static allocation (e.g., allocating space for maximum token lengths at a time). While these strategies effectively increase batch size, they cannot be directly implemented in PIM designs due to PIM's requirement for precise data layout across columns, rows, and banks.

In AttenPIM, for score computation ($Q \times K_t$), to ensure all-bank parallelism, the same row and same columns of all banks in the channel should be allocated when existing memory space is exhausted. For context computation ($S \times V$), N_{BB} banks are employed per token head. Similarly, we allocate the same entire row of these banks as needed.

TABLE I
DYNAMIC ALLOCATION METHOD COMPARISON

	K Cache	V Cache
NeuPIMs	$N_{bank} \times N_{row} / D_{he}$	N_{row} / N_{pre}
AttAcc	$N_{ch} \times N_{bank} \times N_{row} / D_{he}$	$N_{ch} \times N_{bank} \times N_{row} / D_{he}$
AttenPIM	$N_{bank} \times N_{row} / D_{he}$	N_{row} / N_{bl}

Table I compares the number of tokens per dynamic allocation (parameters in Section IV). For K Cache allocation, NeuPIMs and AttenPIM allocate the same entire row (size of N_{row}) of all banks when needed, with an allocation size of $N_{bank} \times N_{row} / D_{he}$ tokens. Since AttAcc maps heads across the entire HBM device, it requires $N_{ch} \times$ memory space. For V Cache allocation, NeuPIMs allocates V cache of N_{row} / N_{pre} tokens at a time, which significantly wastes memory space. AttAcc leverages a transposed data layout of K Cache with the same allocation size. AttenPIM allocates entire row of D_{he} / N_{bl} banks, which total allocation size is N_{row} / N_{bl} .

2) *Kernel Fusion*: NeuPIMs and AttAcc overlap softmax execution time between heads by executing GEMV operations in the PUs while performing softmax operations in softmax unit (SU, either in xPU or PIM) for different heads. To achieve this, NeuPIMs introduces double row buffers in each bank to enable concurrent access from PU and SU, while AttAcc implements direct data transfer from PU buffer to SU without interrupting PU execution. AttenPIM adopts AttAcc’s approach for kernel fusion, achieving enhanced benefits due to its context operations within bank PUs, which reserves all TSV bandwidth for softmax operation transfers.

IV. EVALUATION

A. Experiment Setup

1) *Baselines*: We compare AttenPIM with two state-of-the-art PIM architectures: NeuPIMs [6] and AttAcc [5]. While these baselines provide comprehensive xPU-PIM heterogeneous acceleration for LLMs, AttenPIM specifically targets attention GEMV operations, which constitute the primary performance bottleneck. We leverage HBM as the memory device, with configurations detailed in Table II. To ensure a fair comparison, we maintain consistent hardware parameters across all implementations, including channel count, bank group count, bank count, and so on.

2) *LLM Workloads*: Our evaluation employs four variants of GPT-3 [18], the Large Language Model developed by OpenAI, as detailed in Table III. Since various LLM variants (including other architectures such as Llama [19]) differ primarily in their number of heads while maintaining consistent head dimensions, the attention performance metrics (e.g., execution time, energy consumption) scale approximately linearly with the number of heads. Consequently, we normalize the results relative to the baseline and present the averaged measurements.

For comprehensive performance assessment, we utilize four diverse real-world LLM inference datasets: Alpaca [20], ShareGPT [21], MultiNews [22], and LCC [22]. Alpaca

TABLE II
SIMULATOR DETAILS

HBM Parameters			
Banks/BG	4	Banks/Channel (N_{bank})	32
Burst Length (N_{bl})	32B	Row Buffer Size (N_{row})	2KB
Channels (N_{ch})	8	Capacity/Channel	1GB
Address Mapping	Row-Channel-Rank-Bank-BG-Column		
DRAM Timing Parameters	CCD_S=1:CCD_L=2:RRD=6:RCD=14:RAS=34:RP=14 CL=14:WL=4:WR=16:RTP_L=6:RTP_S=4:RFC=260		
Bank PU Parameters			
Multipliers	16	Adders	16
Frequency	666MHz	Buffer Size	32B

TABLE III
THE EVALUATED LLM CONFIGURATIONS

Model	Heads (N_h)	Embedding(N_e)	Model	Heads (N_h)	Embedding(N_e)
GPT3-7B	32	4096	GPT3-13B	40	5120
GPT3-30B	56	7168	GPT3-175B	96	12288

* Precision: $N_{pre} = 2$ (16-bit floating point); Head Embedding: $D_{he} = N_e / N_h = 128$.

dataset is an instruction dataset generated using OpenAI’s text-davinci-003 engine, characterized by compact token lengths of less than 100 tokens on average. ShareGPT dataset is a set of conversations scraped from the real-world user logs of ChatGPT, with a maximum token length of $2K$ and average of about $0.5K$. Both MultiNews and LCC are from LongBench [22] with average token length of more than $2K$ and $4K$, respectively, which are specialized in long-context processing.

3) *Simulation*: We develop an in-house simulator by modifying DRAMSim3 [23] to evaluate the performance of AttenPIM. We also implement the data layout and execution flow of NeuPIMs and AttAcc in this simulator for comparative analysis. Following NeuPIMs’ methodology, we randomly pick sequence lengths from each dataset for simulation. This way, we can warm up the inference batch so that the batch is filled with requests having various sequence lengths.

For energy consumption, we distinguish memory access at die level (with TSVs) and bank-level (without TSVs), referring to energy consumption reported in previous work [24]. For hardware validation, we implement a prototype of AttenPIM PU with RTL and evaluate it with Synopsys Design Compiler under 28nm CMOS technology.

B. Performance of AttenPIM

We evaluate AttenPIM’s execution time against two baseline architectures, with results normalized to NeuPIMs for each dataset, as illustrated in Fig. 6. The total execution time comprises four distinct components: K preparation, V preparation, score computation ($Q \times K_t$), and context computation ($S \times V$).

First, AttenPIM demonstrates superior performance across all LLM datasets, achieving speedups of $1.20 \times$ – $5.26 \times$ over NeuPIMs and $1.13 \times$ – $3.91 \times$ over AttAcc. Compared to NeuPIMs, AttenPIM takes DRAM operation efficiency into consideration, significantly decreasing the time for V preparation. Compared to AttAcc, AttenPIM constrain all computations within bank PU, eliminating TSV bandwidth constraints.

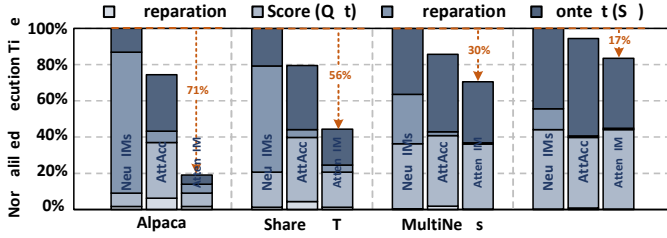


Fig. 6. Normalized performance comparison.

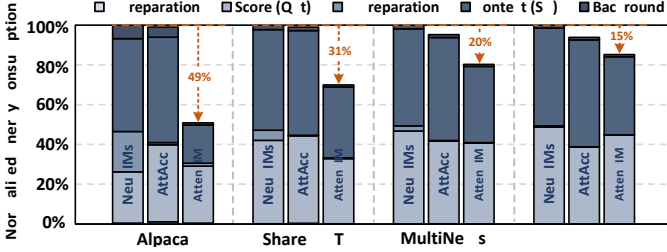


Fig. 7. Normalized energy consumption comparison.

Second, AttenPIM exhibits robust performance across varying token lengths. AttAcc shows worse performance with short token lengths than with long ones due to workload imbalance. NeuPIMs and AttenPIM partition heads to channels instead of HBM devices, whose bank number is usually smaller than token lengths, which results in less workload imbalance with short token lengths.

Third, the performance advantages of AttenPIM show a correlation with dataset token lengths. For individual token attention operations, the relative performance gain decreases as the number of previously generated tokens increases, primarily due to GEMV execution's growing dominance in total execution time. However, since long token sequences are generated incrementally from shorter sequences, AttenPIM's time reductions accumulate at each step, contributing to overall end-to-end inference efficiency.

C. Energy of AttenPIM

We compare the memory access energy consumption of AttenPIM and two baseline architectures. As illustrated in Fig. 6, results are normalized to the highest-consuming baseline for each dataset. The energy analysis is decomposed into five components: K preparation, V preparation, $\text{Score } Q \times K_t$, Context $S \times V$, and background (of entire execution).

First, AttenPIM demonstrates significant energy efficiency improvements, achieving reductions of 14.8%–48.1% and 14.4%–49.2% over NeuPIMs and AttAcc, respectively. The primary energy savings stem from optimized DRAM operations during V cache preparation (over NeuPIMs) and reduced memory accesses on TSVs (over AttAcc). Second, the energy efficiency advantages of AttenPIM also show a correlation with dataset token lengths. The energy waste of masked reads in NeuPIMs and TSV transfers in AttAcc are significant with short token lengths. For long token lengths, the energy for score and context would dominate the overall consumption.

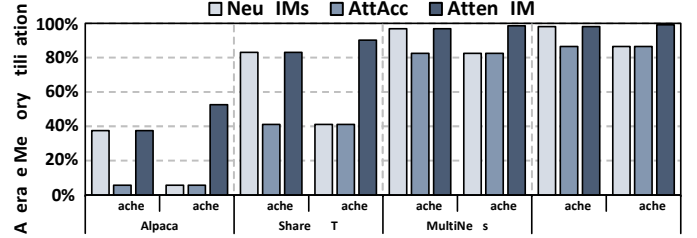


Fig. 8. Memory utilization of allocation method comparison.

TABLE IV
AREA AND POWER OVERHEAD

	NeuPIMs	AttAcc	AttenPIM	HBM
Area (mm^2)	0.79	1.58	0.86	>100 (DRAM process)
Power (W)	0.24	0.26	0.29	116 (Device Budget)

D. Memory Allocation Efficiency

We compare the memory utilization of allocation methods of AttenPIM and two baselines, as shown in Fig. 8. For K cache allocation, both NeuPIMs and AttenPIM demonstrate superior memory utilization compared to AttAcc across all evaluated datasets. This improved efficiency can be attributed to their optimized head distribution strategies. For V cache allocation, AttenPIM achieves the highest memory utilization through the burst block distributed data layout, which achieves memory utilization exceeding 90% as token length increases.

E. Overhead of AttenPIM

We implement bank PU of AttenPIM and two baseline designs in RTL and evaluate them using Synopsys Design Compiler. Based on NeuPIMs' configuration of banks per DRAM die, the result of 64 PUs is shown in Table IV. The evaluation shows that AttenPIM introduces negligible area and power overhead compared to NeuPIM, while achieving a 47% area reduction compared to AttAcc through the elimination of a large input buffer. When considering the scaling of PU area to the third-generation 10nm class (1z-nm) DRAM process, although the area density would be $10\times$ lower than the equivalent logic process, the overhead remains acceptable given the HBM specifications of more than $100mm^2$ DRAM dies and 116W power budget.

V. CONCLUSION

In this paper, we propose AttenPIM, a software-hardware co-design for efficient PIM-based attention acceleration. We propose PIM-oriented data layouts for KV storage and dual-mode computing paradigms tailored for score and context computations, which are supported by a configurable PIM unit design. In addition, we exploit token-level and head-level parallelism to optimize workload balance, while kernel fusion and dynamic allocation methods are proposed to further minimize memory overhead. Experimental results demonstrate that AttenPIM achieves $1.13\times$ – $5.26\times$ speedup and reduces energy consumption by 17%–49% compared to two state-of-the-art PIM baselines.

REFERENCES

- [1] Z. Zhou, X. Ning, K. Hong, T. Fu, J. Xu, S. Li, Y. Lou, L. Wang, Z. Yuan, X. Li *et al.*, “A survey on efficient inference for large language models,” *arXiv preprint arXiv:2404.14294*, 2024.
- [2] H. Lee, G. Kim, D. Yun, I. Kim, Y. Kwon, and E. Lim, “Cost-effective llm accelerator using processing in memory technology,” in *2024 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. IEEE, 2024, pp. 1–2.
- [3] C. Li, Z. Zhou, S. Zheng, J. Zhang, Y. Liang, and G. Sun, “Specpim: Accelerating speculative inference on pim-enabled system via architecture-dataflow co-exploration,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 950–965.
- [4] J. Choi, J. Park, K. Kyung, N. S. Kim, and J. H. Ahn, “Unleashing the potential of pim: Accelerating large batched inference of transformer-based generative models,” *IEEE Computer Architecture Letters*, 2023.
- [5] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, “Attacc! unleashing the power of pim for batched transformer-based generative model inference,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 103–119.
- [6] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, “Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 722–737.
- [7] G. Kim, J. Kim, N. Kim, W. Shin, J. Won, H. Joo, H. Choi, B. An, G. Shin, D. Yun *et al.*, “Sk hynix ai-specific computing memory solution: From aim device to heterogeneous aimx-xpu system for comprehensive llm inference,” in *2024 IEEE Hot Chips 36 Symposium (HCS)*. IEEE Computer Society, 2024, pp. 1–26.
- [8] Y. Kwon, G. Kim, N. Kim, W. Shin, J. Won, H. Joo, H. Choi, B. An, G. Shin, D. Yun *et al.*, “Memory-centric computing with sk hynix’s domain-specific memory,” in *2023 IEEE Hot Chips 35 Symposium (HCS)*. IEEE Computer Society, 2023, pp. 1–26.
- [9] M. He, C. Song, I. Kim, C. Jeong, S. Kim, I. Park, M. Thottethodi, and T. Vijaykumar, “Newton: A dram-maker’s accelerator-in-memory (aim) architecture for machine learning,” in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 372–385.
- [10] Y. Kwon, K. Vladimir, N. Kim, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, G. Kim, B. An *et al.*, “System architecture and software stack for gddr6-aim,” in *2022 IEEE Hot Chips 34 Symposium (HCS)*. IEEE, 2022, pp. 1–25.
- [11] H. Asghari-Moghaddam, Y. H. Son, J. H. Ahn, and N. S. Kim, “Chameleon: Versatile and practical near-dram acceleration architecture for large memory systems,” in *2016 49th annual IEEE/ACM international symposium on Microarchitecture (MICRO)*. IEEE, 2016, pp. 1–13.
- [12] F. Devaux, “The true processing in memory accelerator,” in *2019 IEEE Hot Chips 31 Symposium (HCS)*. IEEE Computer Society, 2019, pp. 1–24.
- [13] H. Kal, C. Yoo, and W. W. Ro, “Aespa: Asynchronous execution scheme to exploit bank-level parallelism of processing-in-memory,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 815–827.
- [14] J. H. Kim, S.-h. Kang, S. Lee, H. Kim, W. Song, Y. Ro, S. Lee, D. Wang, H. Shin, B. Phuah *et al.*, “Aquabolt-xl: Samsung hbm2-pim with in-memory processing for ml accelerators and beyond,” in *2021 IEEE Hot Chips 33 Symposium (HCS)*. IEEE, 2021, pp. 1–26.
- [15] A. Vaswani, “Attention is all you need,” *Advances in Neural Information Processing Systems*, 2017.
- [16] “High Bandwidth Memory DRAM (HBM3).” JEDEC, Standard, 2022.
- [17] B. Hyun, T. Kim, D. Lee, and M. Rhu, “Pathfinding future pim architectures by demystifying a commercial pim technology,” in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 263–279.
- [18] T. B. Brown, “Language models are few-shot learners,” *arXiv preprint arXiv:2005.14165*, 2020.
- [19] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [20] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, “Stanford alpaca: An instruction-following llama model,” 2023.
- [21] “ShareGPT,” [Online], ShareGPT Team, 2023, available: <https://sharegpt.com>.
- [22] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou *et al.*, “Longbench: A bilingual, multitask benchmark for long context understanding,” *arXiv preprint arXiv:2308.14508*, 2023.
- [23] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, “Dramsim3: A cycle-accurate, thermal-capable dram simulator,” *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.
- [24] M. O’Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, “Fine-grained dram: Energy-efficient dram for extreme bandwidth systems,” in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, 2017, pp. 41–54.